

KREACIJSKI PATERNI

Singleton Pattern

Singleton pattern osigurava da određena klasa ima samo jednu instancu i omogućava globalnu tačku pristupa toj instanci.

Upotreba u sistemu: Može se primijeniti na kontroler koji upravlja validacijom proizvoda (ValidacijaProizvoda). Budući da validacija treba biti jedinstvena i konzistentna za sve proizvode, ovaj kontroler bi trebao imati samo jednu instancu.

Prednosti:

- Jedinstvena i globalna tačka pristupa instanci.
- Povećava kontrolu nad zajedničkim resursima.

Implementacija: Dodana statička metoda getInstance() unutar klase ValidacijaProizvoda koja osigurava da postoji samo jedna instanca te klase.

ValidacijaProizvoda
-uniqueInstance : ValidacijaProizvoda
+getInstance() : ValidacijaProizvoda
+provjeriValidnost(id) : bool

Prototype Pattern

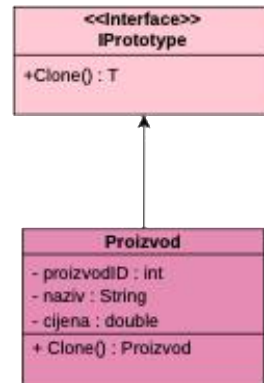
Prototype pattern omogućava kloniranje postojećih objekata umjesto kreiranja novih iz nule. Koristi se kada je kreiranje novih objekata skupo ili složeno.

Upotreba u sistemu: Moguće je primijeniti ovaj pattern na klasu Proizvod. Ako korisnik želi personalizirati proizvod, sistem može klonirati već postojeći proizvod i izvršiti izmjene nad kopijom.

Prednosti:

- Brže kreiranje objekata.
- Omogućava fleksibilnost kod izmjene sličnih objekata.

Implementacija: Klasa Proizvod implementira interfejs Prototip sa metodom kloniraj(), koja vraća kopiju proizvoda.



Factory Method Pattern

Factory Method pattern omogućava da kreiramo objekte preko apstraktne metode bez da eksplicitno navedemo njihovu konkretnu klasu.

Upotreba u sistemu: Pattern je implementiran u kontekstu različitih tipova plaćanja. Umjesto da klasa direktno instancira Placanje, koristi se PlacanjeFactory koja ima metodu kreirajPlacanje().

Prednosti:

- Eliminira zavisnost od konkretnih klasa.
- Omogućava lakše proširenje sistema novim vrstama plaćanja.

Implementacija: Kreirane su klase KarticnoPlacanjeFactory, PaypalPlacanjeFactory koje nasljeđuju PlacanjeFactory i implementiraju metodu kreirajPlacanje() koja vraća konkretan objekat Placanje.

Abstract Factory Pattern

Abstract Factory pattern omogućava kreiranje porodica povezanih objekata bez navođenja konkretnih klasa.

Upotreba u sistemu: Može se primijeniti za kreiranje različitih vrsta proizvoda (npr. buketi, poklon-kutije, igračke). Sistem može imati apstraktnu fabriku ProizvodFabrika sa metodama kreirajBuket(), kreirajPoklonKutiju(), itd.

Prednosti:

- Centralizirano kreiranje porodica proizvoda.
- Omogućava zamjenu čitavih grupa proizvoda bez promjene ostatka koda.

Implementacija: Apstraktna klasa `ProizvodFabrika` sadrži metode za kreiranje proizvoda po kategorijama. Svaka konkretna fabrika implementira metode i vraća konkretne proizvode.

Builder Pattern

Builder pattern koristimo kada imamo složene objekte koje treba kreirati postepeno, u više faza, uz mogućnost da različiti klijenti grade različite reprezentacije istog objekta.

Upotreba u sistemu: U našem sistemu ovaj pattern je primijenjen na klasu **Narudzba**. Budući da narudzba sadrži više međusobno zavisnih komponenti (korisnik, adresa dostave, lista proizvoda, ukupna cijena, status itd.), jasno je da kreiranje ovog objekta zahtijeva više koraka.

Prednosti:

- Pojednostavljuje konstrukciju kompleksnih objekata.
- Omogućava fleksibilnost i veću kontrolu nad procesom instanciranja.

Implementacija: Kreirana je pomoćna klasa `NarudzbaBuilder` koja sadrži metode poput `postaviKorisnika()`, `dodajProizvod()`, `postaviAdresu()` i na kraju `build()` koja vraća gotov objekat tipa `Narudzba`.